# EN3160 Assignment 3

**Gamage S.K.**
**230195F**

GitHub: https://github.com/GamageShakthi/Image-Processing-Assignment-3

## Setup

In [1]:
```
import copy
import os
import random
import time
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.nn import functional as F
from torch.utils.data import DataLoader, TensorDataset
import torchvision
from torchvision import datasets, transforms, models
from torchvision.datasets.utils import download_and_extract_archive
from IPython.display import display, Markdown

SEED = 195
DATA = Path('data')
RESULTS = Path('results_230195F')
DATA.mkdir(exist_ok=True)
RESULTS.mkdir(exist_ok=True)
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
torch.set_num_threads(min(4, os.cpu_count() or 1))

def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = False
    torch.backends.cudnn.deterministic = True

def make_loader(dataset, batch_size, shuffle=False, seed=SEED):
    generator = torch.Generator().manual_seed(seed)
    return DataLoader(dataset, batch_size=batch_size, shuffle=shuffle,
                      num_workers=0, generator=generator,
                      pin_memory=(DEVICE.type == 'cuda'))
```

```
set_seed(SEED)
plt.rcParams.update({'figure.dpi': 110, 'font.size': 10,
                     'axes.spines.top': False, 'axes.spines.right': False})
print('PyTorch:', torch.__version__)
print('Torchvision:', torchvision.__version__)
print('Device:', DEVICE)
print('Random seed:', SEED)
```

```
PyTorch: 2.14.1+cpu
Torchvision: 0.29.1+cpu
Device: cpu
Random seed: 195
```

In [2]:

```
@torch.no_grad()
def evaluate(model, loader, predictions=False):
    if isinstance(model, nn.Module):
        model.eval()
    total_loss, correct, count = 0.0, 0, 0
    actual, predicted = [], []
    for images, labels in loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        scores = model(images)
        total_loss += F.cross_entropy(scores, labels, reduction='sum').item()
        guesses = scores.argmax(1)
        correct += (guesses == labels).sum().item()
        count += labels.numel()
        if predictions:
            actual.append(labels.cpu())
            predicted.append(guesses.cpu())
    result = {'loss': total_loss / count, 'accuracy': 100 * correct / count}
    if predictions:
        result['actual'] = torch.cat(actual).numpy()
        result['predicted'] = torch.cat(predicted).numpy()
    return result

def save_plot(filename):
    plt.tight_layout()
    plt.savefig(RESULTS / filename, dpi=160, bbox_inches='tight')
    plt.show()
    plt.close()

def learning_plot(history, title, split='test'):
    fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))
    for ax, metric, label in zip(axes, ['loss', 'accuracy'],
                                 ['Cross entropy', 'Accuracy (%)']):
        ax.plot(history['epoch'], history['train_' + metric],
                marker='o', markersize=3, label='Training', color='#195e83')
        ax.plot(history['epoch'], history[split + '_' + metric],
                marker='s', markersize=3, label=split.capitalize(), color='#ca6930'
        ax.set(xlabel='Epoch', ylabel=label, title=title + ': ' + label)
        ax.set_xticks(history['epoch'])
        ax.grid(alpha=0.2)
        ax.legend()
    save_plot(title.lower().replace(' ', '_') + '.png')
```

# Question 1

## 1(a) Hidden layer

The network maps 3072 CIFAR10 inputs to 100 sigmoid units and ten class scores.

In [3]:
```
cifar_train_raw = datasets.CIFAR10(str(DATA), train=True, download=True)
cifar_test_raw = datasets.CIFAR10(str(DATA), train=False, download=True)

def cifar_tensors(dataset):
    images = torch.from_numpy(dataset.data).permute(0, 3, 1, 2).contiguous()
    images = images.flatten(1).float().div_(127.5).sub_(1.0)
    targets = torch.tensor(dataset.targets, dtype=torch.long)
    return TensorDataset(images, targets)

cifar_train = cifar_tensors(cifar_train_raw)
cifar_test = cifar_tensors(cifar_test_raw)
cifar_loader = make_loader(cifar_train, 50, shuffle=True)
cifar_train_eval = make_loader(cifar_train, 512)
cifar_test_eval = make_loader(cifar_test, 512)
print('Training samples:', len(cifar_train))
print('Test samples:', len(cifar_test))
print('Input features:', cifar_train.tensors[0].shape[1])
```

```
Training samples: 50000
Test samples: 10000
Input features: 3072
```

## 1(b) Cross entropy

The loss and gradients are computed manually using stable log probabilities.

In [4]:
```
set_seed(SEED)
sizes = (3072, 100, 10)
parameters = {
    'W1': torch.randn(sizes[0], sizes[1], device=DEVICE) * np.sqrt(2 / 3172),
    'b1': torch.zeros(sizes[1], device=DEVICE),
    'W2': torch.randn(sizes[1], sizes[2], device=DEVICE) * np.sqrt(2 / 110),
    'b2': torch.zeros(sizes[2], device=DEVICE)
}

def dense_forward(inputs):
    hidden = torch.sigmoid(inputs @ parameters['W1'] + parameters['b1'])
    scores = hidden @ parameters['W2'] + parameters['b2']
    return hidden, scores

def dense_scores(inputs):
    return dense_forward(inputs)[1]

@torch.no_grad()
def dense_update(inputs, labels, learning_rate):
    hidden, scores = dense_forward(inputs)
```

```
    log_probabilities = scores - torch.logsumexp(scores, dim=1, keepdim=True)
    rows = torch.arange(labels.numel(), device=DEVICE)
    loss = -log_probabilities[rows, labels].mean()

    # Compute every gradient before changing either weight matrix.
    score_gradient = log_probabilities.exp()
    score_gradient[rows, labels] -= 1
    score_gradient /= labels.numel()
    hidden_gradient = (score_gradient @ parameters['W2'].T) * hidden * (1 - hidden)
    gradients = {
        'W1': inputs.T @ hidden_gradient,
        'b1': hidden_gradient.sum(0),
        'W2': hidden.T @ score_gradient,
        'b2': score_gradient.sum(0)
    }
    for name in parameters:
        parameters[name].add_(gradients[name], alpha=-learning_rate)
    return loss.item()

print('Network:', sizes)
print('Parameters:', sum(p.numel() for p in parameters.values()))
```

```
Network: (3072, 100, 10)
Parameters: 308310
```

## 1(c) Training

I used Xavier initialization and ten epochs, with a learning rate starting at 0.1.

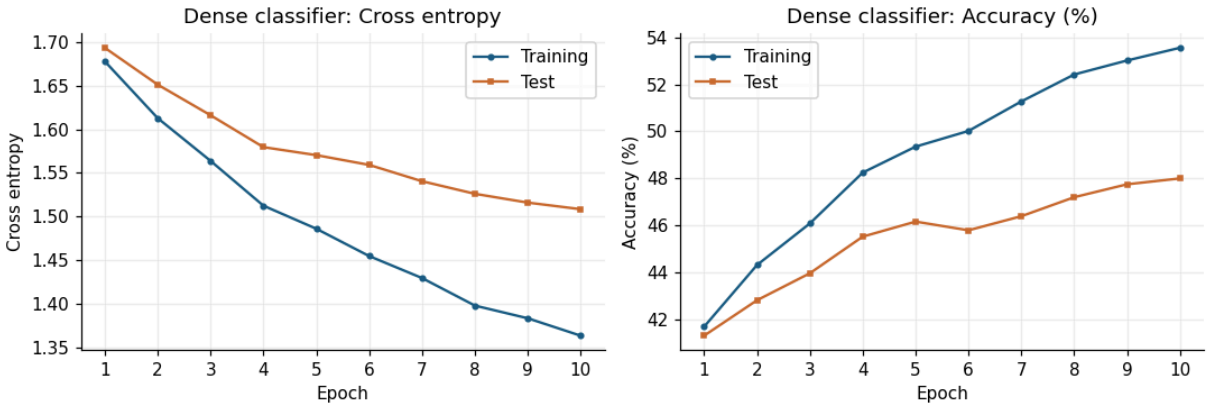
In [5]:

```
dense_rows = []
for epoch in range(1, 11):
    rate = 0.1 * (0.9 ** (epoch - 1))
    for images, labels in cifar_loader:
        dense_update(images.to(DEVICE), labels.to(DEVICE), rate)
    train_metrics = evaluate(dense_scores, cifar_train_eval)
    test_metrics = evaluate(dense_scores, cifar_test_eval)
    dense_rows.append({
        'epoch': epoch, 'learning_rate': rate,
        'train_loss': train_metrics['loss'], 'test_loss': test_metrics['loss'],
        'train_accuracy': train_metrics['accuracy'],
        'test_accuracy': test_metrics['accuracy']
    })
    print(f"Epoch {epoch:2d}/10  Training {train_metrics['accuracy']:.2f}%  "
          f"Test {test_metrics['accuracy']:.2f}%  Test loss {test_metrics['loss']:.

dense_history = pd.DataFrame(dense_rows)
display(dense_history.round(4))
dense_history.to_csv(RESULTS / 'q1_history.csv', index=False)
learning_plot(dense_history, 'Dense classifier')
```

```
Epoch  1/10  Training 41.67%  Test 41.29%  Test loss 1.6935
Epoch  2/10  Training 44.30%  Test 42.80%  Test loss 1.6513
Epoch  3/10  Training 46.07%  Test 43.95%  Test loss 1.6161
Epoch  4/10  Training 48.24%  Test 45.51%  Test loss 1.5795
Epoch  5/10  Training 49.34%  Test 46.15%  Test loss 1.5703
Epoch  6/10  Training 50.01%  Test 45.78%  Test loss 1.5591
Epoch  7/10  Training 51.27%  Test 46.38%  Test loss 1.5404
Epoch  8/10  Training 52.42%  Test 47.19%  Test loss 1.5259
Epoch  9/10  Training 53.01%  Test 47.74%  Test loss 1.5158
Epoch 10/10  Training 53.56%  Test 47.99%  Test loss 1.5083
```

| | epoch | learning_rate | train_loss | test_loss | train_accuracy | test_accuracy |
|---|---|---|---|---|---|---|
| **0** | 1 | 0.1000 | 1.6780 | 1.6935 | 41.672 | 41.29 |
| **1** | 2 | 0.0900 | 1.6128 | 1.6513 | 44.300 | 42.80 |
| **2** | 3 | 0.0810 | 1.5638 | 1.6161 | 46.072 | 43.95 |
| **3** | 4 | 0.0729 | 1.5123 | 1.5795 | 48.238 | 45.51 |
| **4** | 5 | 0.0656 | 1.4859 | 1.5703 | 49.344 | 46.15 |
| **5** | 6 | 0.0590 | 1.4545 | 1.5591 | 50.012 | 45.78 |
| **6** | 7 | 0.0531 | 1.4293 | 1.5404 | 51.270 | 46.38 |
| **7** | 8 | 0.0478 | 1.3977 | 1.5259 | 52.418 | 47.19 |
| **8** | 9 | 0.0430 | 1.3831 | 1.5158 | 53.014 | 47.74 |
| **9** | 10 | 0.0387 | 1.3631 | 1.5083 | 53.556 | 47.99 |




Training accuracy was **53.56%**, compared with **47.99%** on test images. The gap suggests some overfitting.

# Question 2

Both models use the same MNIST preprocessing, batch size 128, Adam with learning rate 0.001, and ten epochs. All scores are measured in evaluation mode.

In [7]:
```
mnist_train_raw = datasets.MNIST(str(DATA), train=True, download=True)
mnist_test_raw = datasets.MNIST(str(DATA), train=False, download=True)
```

```
def mnist_tensors(dataset):
    images = dataset.data.unsqueeze(1).float().div(255)
    images = F.pad(images, (2, 2, 2, 2))
    images = (images - 0.1307) / 0.3081
    return TensorDataset(images, dataset.targets.clone())

mnist_train = mnist_tensors(mnist_train_raw)
mnist_test = mnist_tensors(mnist_test_raw)
mnist_train_eval = make_loader(mnist_train, 512)
mnist_test_eval = make_loader(mnist_test, 512)
print('Training samples:', len(mnist_train))
print('Test samples:', len(mnist_test))
print('Image shape:', tuple(mnist_train.tensors[0].shape[1:]))
```

```
Training samples: 60000
Test samples: 10000
Image shape: (1, 32, 32)
```

## 2(a) LeNet5

This adaptation uses tanh and average pooling.

In [8]:
```
class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.c1 = nn.Conv2d(1, 6, 5)
        self.c3 = nn.Conv2d(6, 16, 5)
        self.c5 = nn.Conv2d(16, 120, 5)
        self.f6 = nn.Linear(120, 84)
        self.output = nn.Linear(84, 10)

    def forward(self, images):
        x = F.avg_pool2d(torch.tanh(self.c1(images)), 2)
        x = F.avg_pool2d(torch.tanh(self.c3(x)), 2)
        x = torch.tanh(self.c5(x)).flatten(1)
        return self.output(torch.tanh(self.f6(x)))

print(LeNet5())
```

```
LeNet5(
  (c1): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1))
  (c3): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
  (c5): Conv2d(16, 120, kernel_size=(5, 5), stride=(1, 1))
  (f6): Linear(in_features=120, out_features=84, bias=True)
  (output): Linear(in_features=84, out_features=10, bias=True)
)
```

In [9]:
```
def train_epoch(model, loader, optimizer, frozen_backbone=False):
    if frozen_backbone:
        # Keep both pretrained weights and BatchNorm statistics fixed.
        model.eval()
        model.fc.train()
    else:
        model.train()
    for images, labels in loader:
```

```
        images, labels = images.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad(set_to_none=True)
        loss = F.cross_entropy(model(images), labels)
        loss.backward()
        optimizer.step()

def fit_digits(constructor, name):
    set_seed(SEED)
    model = constructor().to(DEVICE)
    training = make_loader(mnist_train, 128, shuffle=True)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    rows = []
    started = time.perf_counter()
    for epoch in range(1, 11):
        train_epoch(model, training, optimizer)
        train = evaluate(model, mnist_train_eval)
        test = evaluate(model, mnist_test_eval)
        rows.append({'epoch': epoch,
                     'train_loss': train['loss'], 'test_loss': test['loss'],
                     'train_accuracy': train['accuracy'],
                     'test_accuracy': test['accuracy']})
        print(f"{name}  {epoch:2d}/10  "
              f"Train loss {train['loss']:.4f}  Test loss {test['loss']:.4f}  "
              f"Train {train['accuracy']:.2f}%  Test {test['accuracy']:.2f}%")
    history = pd.DataFrame(rows)
    history.to_csv(RESULTS / (name.lower() + '_history.csv'), index=False)
    torch.save(model.state_dict(), RESULTS / (name.lower() + '.pt'))
    return model, history, time.perf_counter() - started
```

In [10]:
```
lenet_model, lenet_history, lenet_seconds = fit_digits(LeNet5, 'LeNet5')
```

```
LeNet5   1/10  Train loss 0.1149  Test loss 0.1133  Train 96.55%  Test 96.61%
LeNet5   2/10  Train loss 0.0614  Test loss 0.0647  Train 98.21%  Test 98.02%
LeNet5   3/10  Train loss 0.0426  Test loss 0.0462  Train 98.73%  Test 98.54%
LeNet5   4/10  Train loss 0.0387  Test loss 0.0523  Train 98.80%  Test 98.33%
LeNet5   5/10  Train loss 0.0240  Test loss 0.0377  Train 99.27%  Test 98.83%
LeNet5   6/10  Train loss 0.0214  Test loss 0.0360  Train 99.36%  Test 98.85%
LeNet5   7/10  Train loss 0.0265  Test loss 0.0441  Train 99.13%  Test 98.56%
LeNet5   8/10  Train loss 0.0161  Test loss 0.0364  Train 99.52%  Test 98.86%
LeNet5   9/10  Train loss 0.0099  Test loss 0.0336  Train 99.74%  Test 98.94%
LeNet5  10/10  Train loss 0.0116  Test loss 0.0388  Train 99.66%  Test 98.87%
```

## 2(b) Small CNN

The three convolutional layers use batch normalization, with dropout before the output.

In [11]:
```
class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, 3, padding=1, bias=False)
        self.norm1 = nn.BatchNorm2d(16)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1, bias=False)
        self.norm2 = nn.BatchNorm2d(32)
        self.conv3 = nn.Conv2d(32, 64, 3, padding=1, bias=False)
        self.norm3 = nn.BatchNorm2d(64)
```

```
        self.hidden = nn.Linear(64 * 4 * 4, 64)
        self.dropout = nn.Dropout(0.4)
        self.output = nn.Linear(64, 10)

    def forward(self, images):
        x = F.max_pool2d(F.relu(self.norm1(self.conv1(images))), 2)
        x = F.max_pool2d(F.relu(self.norm2(self.conv2(x))), 2)
        x = F.max_pool2d(F.relu(self.norm3(self.conv3(x))), 2)
        x = F.relu(self.hidden(x.flatten(1)))
        return self.output(self.dropout(x))

print(SmallCNN())
```

```
SmallCNN(
  (conv1): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=Fal
se)
  (norm1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, bias=True, track_ru
nning_stats=True)
  (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=Fa
lse)
  (norm2): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, bias=True, track_ru
nning_stats=True)
  (conv3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=Fa
lse)
  (norm3): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, bias=True, track_ru
nning_stats=True)
  (hidden): Linear(in_features=1024, out_features=64, bias=True)
  (dropout): Dropout(p=0.4, inplace=False)
  (output): Linear(in_features=64, out_features=10, bias=True)
)
```

In [12]:
```
cnn_model, cnn_history, cnn_seconds = fit_digits(SmallCNN, 'Small_CNN')
```

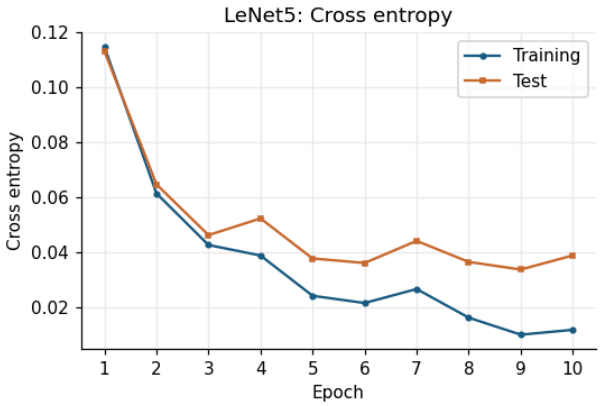
```
Small_CNN   1/10  Train loss 0.0599  Test loss 0.0540  Train 98.12%  Test 98.20%
Small_CNN   2/10  Train loss 0.0383  Test loss 0.0425  Train 98.83%  Test 98.72%
Small_CNN   3/10  Train loss 0.0321  Test loss 0.0339  Train 99.01%  Test 98.93%
Small_CNN   4/10  Train loss 0.0226  Test loss 0.0326  Train 99.28%  Test 98.95%
Small_CNN   5/10  Train loss 0.0157  Test loss 0.0259  Train 99.52%  Test 99.13%
Small_CNN   6/10  Train loss 0.0207  Test loss 0.0301  Train 99.30%  Test 98.92%
Small_CNN   7/10  Train loss 0.0199  Test loss 0.0335  Train 99.36%  Test 98.98%
Small_CNN   8/10  Train loss 0.0116  Test loss 0.0247  Train 99.63%  Test 99.21%
Small_CNN   9/10  Train loss 0.0107  Test loss 0.0305  Train 99.65%  Test 98.97%
Small_CNN  10/10  Train loss 0.0129  Test loss 0.0311  Train 99.59%  Test 99.07%
```

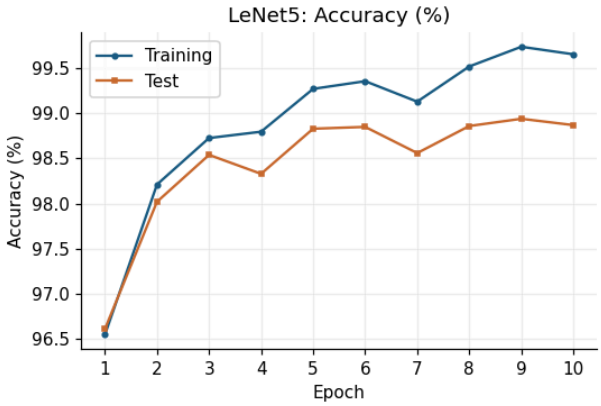
## 2(c) Results

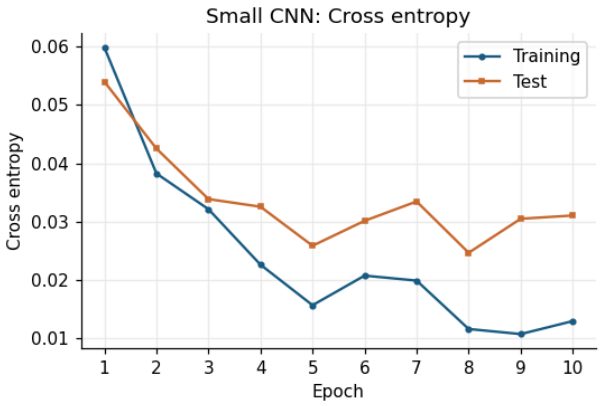
In [13]:
```
mnist_runs = [('LeNet5', lenet_model, lenet_history, lenet_seconds),
              ('Small CNN', cnn_model, cnn_history, cnn_seconds)]
comparison_rows = []
for name, model, history, seconds in mnist_runs:
    last = history.iloc[-1]
    comparison_rows.append({
        'Model': name,
        'Parameters': sum(p.numel() for p in model.parameters()),
        'Train accuracy (%)': last.train_accuracy,
        'Test accuracy (%)': last.test_accuracy,
```

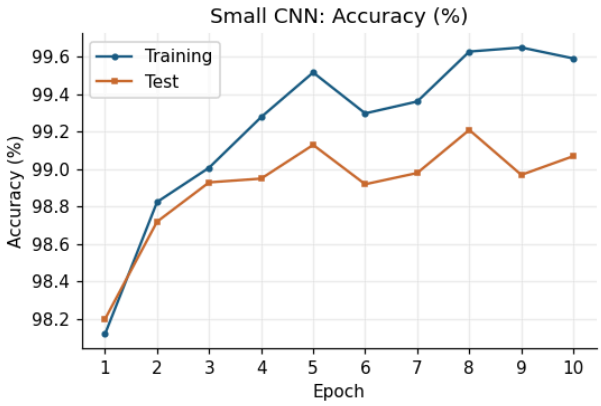
```
        'Train loss': last.train_loss, 'Test loss': last.test_loss,
        'Time (s)': seconds
    })
mnist_results = pd.DataFrame(comparison_rows).set_index('Model')
display(mnist_results.round(4))
mnist_results.to_csv(RESULTS / 'q2_comparison.csv')
learning_plot(lenet_history, 'LeNet5')
learning_plot(cnn_history, 'Small CNN')
```

| Model | Parameters | Train accuracy (%) | Test accuracy (%) | Train loss | Test loss | Time (s) |
|---|---|---|---|---|---|---|
| **LeNet5** | 61706 | 99.6550 | 98.87 | 0.0116 | 0.0388 | 73.5465 |
| **Small CNN** | 89658 | 99.5917 | 99.07 | 0.0129 | 0.0311 | 211.4219 |



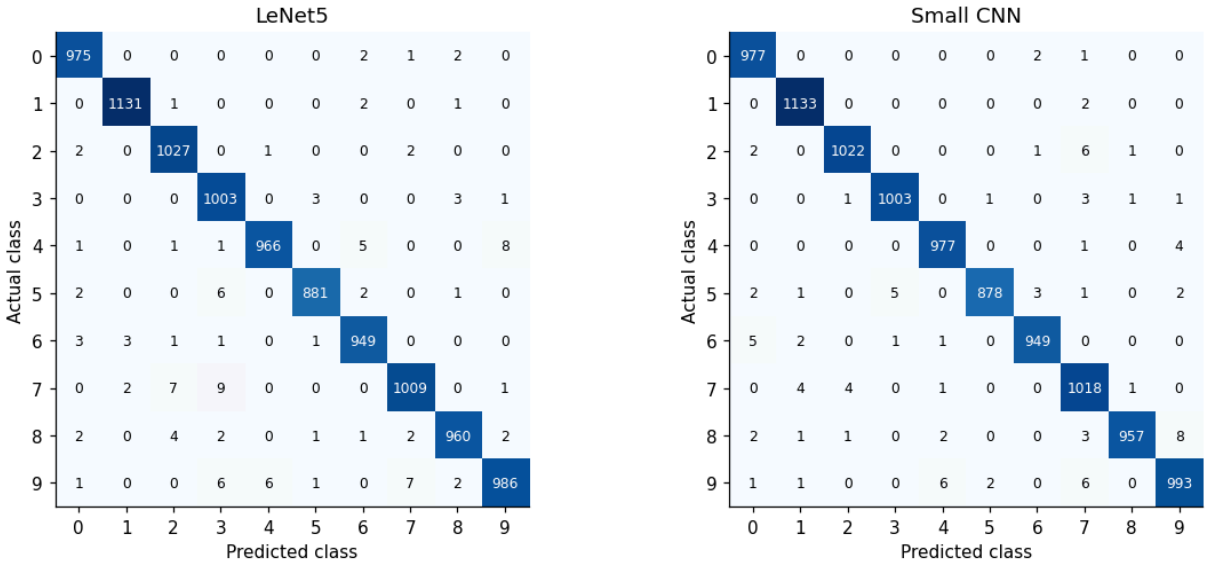




In [14]:
```
def confusion_table(actual, predicted, classes):
    counts = np.zeros((classes, classes), dtype=int)
    np.add.at(counts, (actual, predicted), 1)
    return counts

def plot_confusion(ax, matrix, names, title):
    ax.imshow(matrix, cmap='Blues')
    ax.set(xticks=range(len(names)), yticks=range(len(names)),
           xticklabels=names, yticklabels=names,
           xlabel='Predicted class', ylabel='Actual class', title=title)
    threshold = matrix.max() / 2
```

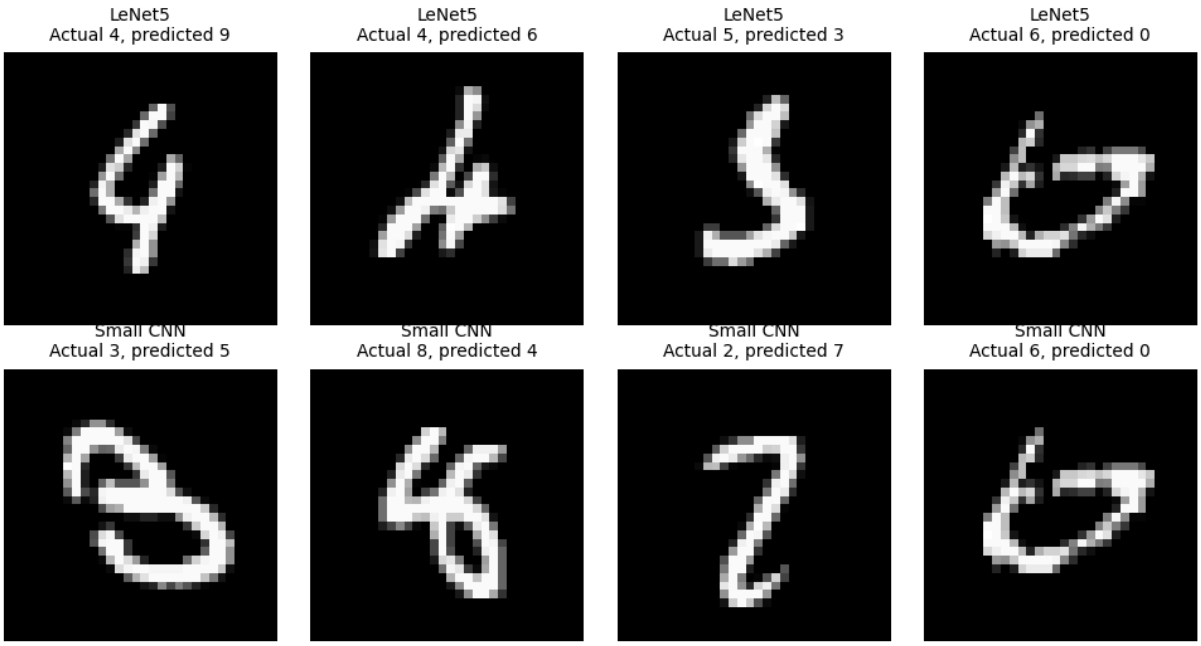
```
    for row in range(len(names)):
        for column in range(len(names)):
            ax.text(column, row, str(matrix[row, column]), ha='center', va='center'
                    fontsize=8, color='white' if matrix[row, column] > threshold el

digit_predictions, digit_matrices = {}, {}
fig, axes = plt.subplots(1, 2, figsize=(11, 4.7))
for ax, (name, model, _, _) in zip(axes, mnist_runs):
    result = evaluate(model, mnist_test_eval, predictions=True)
    matrix = confusion_table(result['actual'], result['predicted'], 10)
    assert matrix.sum() == len(mnist_test)
    digit_predictions[name] = result
    digit_matrices[name] = matrix
    plot_confusion(ax, matrix, list(range(10)), name)
    pd.DataFrame(matrix).to_csv(RESULTS / (name.lower().replace(' ', '_') + '_confu
save_plot('q2_confusion.png')
```




In [15]:

```
fig, axes = plt.subplots(2, 4, figsize=(9, 4.8))
for row, (name, result) in enumerate(digit_predictions.items()):
    mistakes = np.flatnonzero(result['actual'] != result['predicted'])[:4]
    for ax in axes[row]:
        ax.axis('off')
    for ax, index in zip(axes[row], mistakes):
        image = mnist_test.tensors[0][index, 0] * 0.3081 + 0.1307
        ax.imshow(image, cmap='gray', vmin=0, vmax=1)
        ax.set_title(f"{name}\nActual {result['actual'][index]}, predicted {result[
                     fontsize=9)
save_plot('q2_mistakes.png')
```




The small CNN did slightly better, making 20 fewer test errors. LeNet5 most often mistook 7 for 3, while the small CNN confused 8 with 9.

## Question 3

ImageNet1K ResNet18 is adapted to ants and bees. Both methods use the same initialization, augmentation and optimizer settings for ten epochs.

In [17]:
```
insect_root = DATA / 'hymenoptera_data'
if not all((insect_root / split / category).is_dir()
           for split in ['train', 'val'] for category in ['ants', 'bees']):
    download_and_extract_archive(
        'https://download.pytorch.org/tutorial/hymenoptera_data.zip',
        download_root=str(DATA), extract_root=str(DATA)
    )

image_mean = [0.485, 0.456, 0.406]
image_std = [0.229, 0.224, 0.225]
training_transform = transforms.Compose([
    transforms.RandomResizedCrop(224), transforms.RandomHorizontalFlip(),
    transforms.ToTensor(), transforms.Normalize(image_mean, image_std)
])
evaluation_transform = transforms.Compose([
    transforms.Resize(256), transforms.CenterCrop(224),
    transforms.ToTensor(), transforms.Normalize(image_mean, image_std)
])
insect_train = datasets.ImageFolder(str(insect_root / 'train'), training_transform)
insect_train_clean = datasets.ImageFolder(str(insect_root / 'train'), evaluation_tr
insect_validation = datasets.ImageFolder(str(insect_root / 'val'), evaluation_trans
class_names = insect_train.classes
assert insect_validation.class_to_idx == insect_train.class_to_idx
insect_train_eval = make_loader(insect_train_clean, 32)
insect_val_eval = make_loader(insect_validation, 32)
```

```
print('Classes:', class_names)
print('Training images:', len(insect_train))
print('Validation images:', len(insect_validation))
```

```
Classes: ['ants', 'bees']
Training images: 244
Validation images: 153
```

In [18]:
```
def make_resnet(freeze_features):
    set_seed(SEED)
    model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
    model.fc = nn.Linear(model.fc.in_features, len(class_names))
    if freeze_features:
        for name, parameter in model.named_parameters():
            parameter.requires_grad_(name.startswith('fc.'))
    return model.to(DEVICE)
```

In [19]:
```
TRANSFER_EPOCHS = 10

def fit_transfer(freeze_features, label):
    model = make_resnet(freeze_features)
    trainable = [p for p in model.parameters() if p.requires_grad]
    optimizer = torch.optim.SGD(trainable, lr=0.001, momentum=0.9)
    scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=7, gamma=0.1)
    loader = make_loader(insect_train, 4, shuffle=True)
    history_rows = []
    best_accuracy, best_epoch, best_weights = -1.0, 0, None
    frozen_start = None
    if freeze_features:
        frozen_start = {k: v.detach().cpu().clone() for k, v in model.state_dict().
                        if not k.startswith('fc.')}
    started = time.perf_counter()
    for epoch in range(1, TRANSFER_EPOCHS + 1):
        rate = optimizer.param_groups[0]['lr']
        train_epoch(model, loader, optimizer, frozen_backbone=freeze_features)
        train = evaluate(model, insect_train_eval)
        validation = evaluate(model, insect_val_eval)
        history_rows.append({
            'epoch': epoch, 'learning_rate': rate,
            'train_loss': train['loss'], 'val_loss': validation['loss'],
            'train_accuracy': train['accuracy'], 'val_accuracy': validation['accura
        })
        if validation['accuracy'] > best_accuracy:
            best_accuracy, best_epoch = validation['accuracy'], epoch
            best_weights = {k: v.detach().cpu().clone() for k, v in model.state_dic
        scheduler.step()
        print(f"{label}  {epoch:2d}/{TRANSFER_EPOCHS}  "
              f"Train {train['accuracy']:.2f}%  Validation {validation['accuracy']:
              f"Validation loss {validation['loss']:.4f}")
    if freeze_features:
        assert all(torch.equal(before, model.state_dict()[key].cpu())
                   for key, before in frozen_start.items())
        print('Pretrained weights and batch statistics remained fixed.')
    seconds = time.perf_counter() - started
    history = pd.DataFrame(history_rows)
    stem = label.lower().replace(' ', '_')
```

```
    history.to_csv(RESULTS / (stem + '_history.csv'), index=False)
    model.load_state_dict(best_weights)
    torch.save({'state_dict': best_weights, 'classes': class_names, 'epoch': best_e
                RESULTS / (stem + '_best.pt'))
    best_result = evaluate(model, insect_val_eval, predictions=True)
    summary = {
        'Method': label, 'Trainable parameters': sum(p.numel() for p in trainable),
        'Final train (%)': history.iloc[-1].train_accuracy,
        'Final validation (%)': history.iloc[-1].val_accuracy,
        'Best validation (%)': best_result['accuracy'],
        'Best epoch': best_epoch, 'Time (s)': seconds
    }
    return model, history, best_result, summary
```

## 3(a) Fine tuning

In [20]:
```
fine_model, fine_history, fine_result, fine_summary = fit_transfer(False, 'Fine tun
```

```
Fine tuning   1/10  Train 94.67%  Validation 93.46%  Validation loss 0.1881
Fine tuning   2/10  Train 88.52%  Validation 87.58%  Validation loss 0.3687
Fine tuning   3/10  Train 95.08%  Validation 90.20%  Validation loss 0.3539
Fine tuning   4/10  Train 89.34%  Validation 78.43%  Validation loss 0.4919
Fine tuning   5/10  Train 89.34%  Validation 84.31%  Validation loss 0.4428
Fine tuning   6/10  Train 93.03%  Validation 81.05%  Validation loss 0.6854
Fine tuning   7/10  Train 96.31%  Validation 91.50%  Validation loss 0.3058
Fine tuning   8/10  Train 97.54%  Validation 88.89%  Validation loss 0.2795
Fine tuning   9/10  Train 98.36%  Validation 91.50%  Validation loss 0.2175
Fine tuning  10/10  Train 98.36%  Validation 90.20%  Validation loss 0.2435
```

## 3(b) Feature extraction

Only the final classifier learns. Batch normalization statistics remain fixed.

In [21]:
```
fixed_model, fixed_history, fixed_result, fixed_summary = fit_transfer(True, 'Featu
```

```
Feature extraction   1/10  Train 96.31%  Validation 96.08%  Validation loss 0.1265
Feature extraction   2/10  Train 95.90%  Validation 93.46%  Validation loss 0.1995
Feature extraction   3/10  Train 98.36%  Validation 94.77%  Validation loss 0.1480
Feature extraction   4/10  Train 97.54%  Validation 95.42%  Validation loss 0.1396
Feature extraction   5/10  Train 97.13%  Validation 94.77%  Validation loss 0.1393
Feature extraction   6/10  Train 98.77%  Validation 95.42%  Validation loss 0.1437
Feature extraction   7/10  Train 99.18%  Validation 95.42%  Validation loss 0.1474
Feature extraction   8/10  Train 99.18%  Validation 95.42%  Validation loss 0.1483
Feature extraction   9/10  Train 99.59%  Validation 95.42%  Validation loss 0.1515
Feature extraction  10/10  Train 99.18%  Validation 95.42%  Validation loss 0.1454
Pretrained weights and batch statistics remained fixed.
```

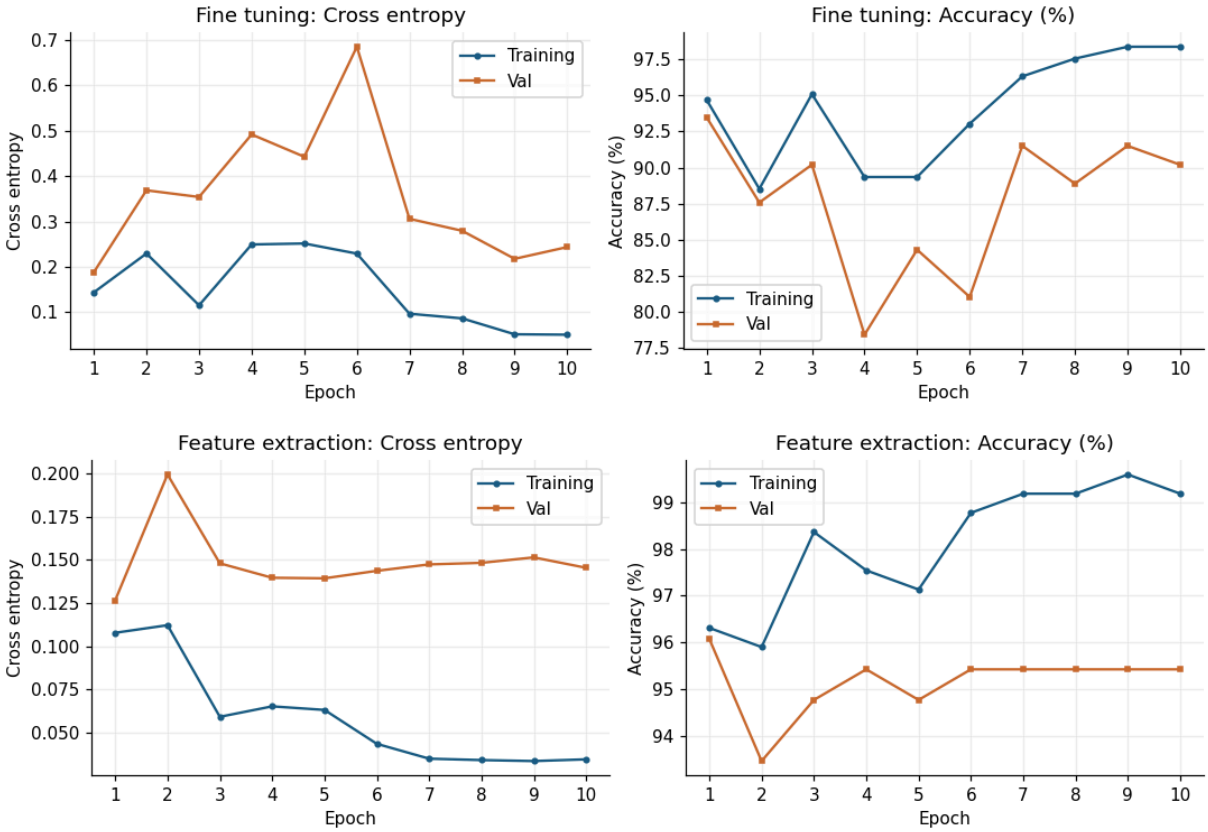
## Results

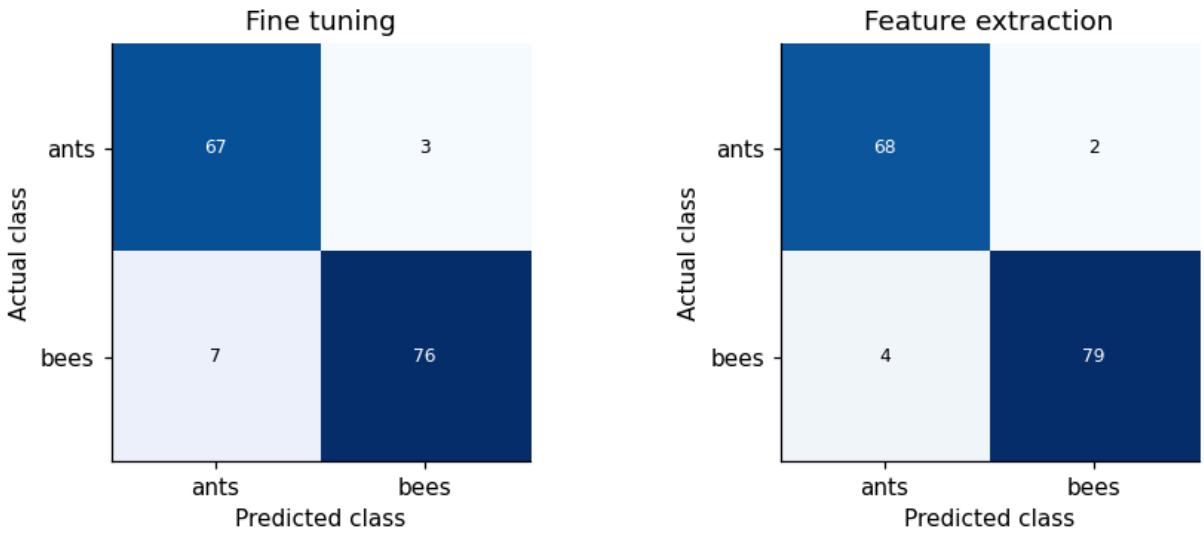
In [22]:
```
transfer_results = pd.DataFrame([fine_summary, fixed_summary]).set_index('Method')
display(transfer_results.round(3))
transfer_results.to_csv(RESULTS / 'q3_comparison.csv')
learning_plot(fine_history, 'Fine tuning', split='val')
```

```
learning_plot(fixed_history, 'Feature extraction', split='val')

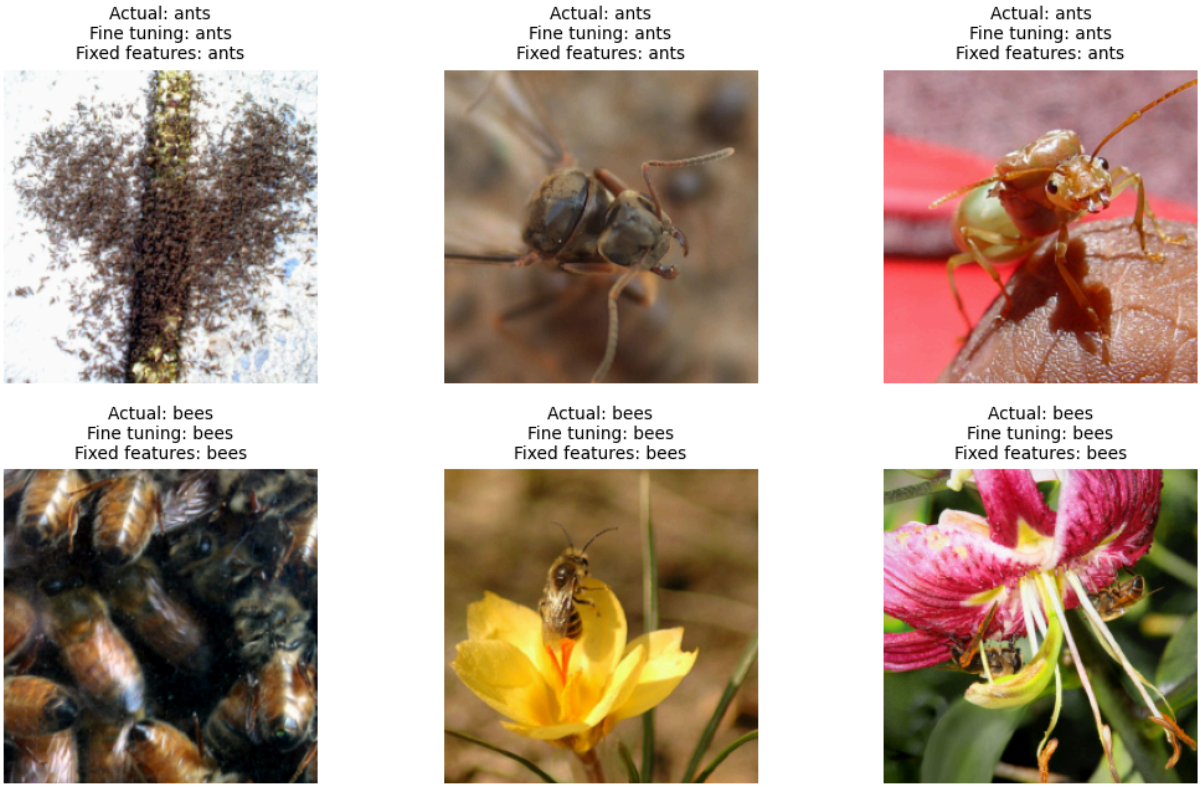
fig, axes = plt.subplots(1, 2, figsize=(8.5, 3.5))
for ax, result, label in zip(axes, [fine_result, fixed_result],
                             ['Fine tuning', 'Feature extraction']):
    matrix = confusion_table(result['actual'], result['predicted'], len(class_names
    assert matrix.sum() == len(insect_validation)
    plot_confusion(ax, matrix, class_names, label)
save_plot('q3_confusion.png')
```

| Method | Trainable parameters | Final train (%) | Final validation (%) | Best validation (%) | Best epoch | Time (s) |
|---|---|---|---|---|---|---|
| **Fine tuning** | 11177538 | 98.361 | 90.196 | 93.464 | 1 | 246.738 |
| **Feature extraction** | 1026 | 99.180 | 95.425 | 96.078 | 1 | 147.844 |






In [23]:
```
targets = np.array(insect_validation.targets)
sample_indices = np.concatenate([np.flatnonzero(targets == label)[:3]
                                 for label in range(len(class_names))])
fig, axes = plt.subplots(2, 3, figsize=(10, 6))
for ax, index in zip(axes.flat, sample_indices):
    tensor, label = insect_validation[int(index)]
    rgb = tensor.permute(1, 2, 0).numpy()
    rgb = np.clip(rgb * image_std + image_mean, 0, 1)
    ax.imshow(rgb)
    ft_name = class_names[fine_result['predicted'][index]]
    fx_name = class_names[fixed_result['predicted'][index]]
    ax.set_title(f'Actual: {class_names[label]}\nFine tuning: {ft_name}\nFixed feat
                 fontsize=9)
    ax.axis('off')
save_plot('q3_predictions.png')
```




Fixed features reached **96.08%** validation accuracy, compared with **93.46%** for fine tuning. The pretrained features worked well with only the classifier trained. These scores were used to select checkpoints, so they are not independent test results.

## Thank You

In [ ]: